

Make My Wedding Plan

System Design Architecture Document

Version 1.0 • V1 Architecture Baseline

Architecture: Modular Monolith | Frontend: Next.js | Backend: Express.js + TypeScript |
Database: MongoDB Atlas

1. Document Overview

This document defines the initial system architecture for Make My Wedding Plan, a private, invitation-only wedding-management platform designed for Indian weddings. It converts the approved product and architecture decisions into an implementation-oriented baseline.

Item	Decision
Product	Make My Wedding Plan
Architecture style	Modular monolith
Frontend	Next.js + TypeScript
Backend	Node.js + Express.js + TypeScript
Database	MongoDB Atlas with Mongoose
Deployment	Two separate Vercel projects
Photo storage	Private Cloudflare R2
Email provider	Resend API
API style	Versioned REST + JSON (/api/v1)
Authentication	JWT access and refresh tokens in secure HttpOnly cookies

2. Product Context

The platform enables wedding families to manage wedding information, events, guests, RSVPs, photos, tasks, vendors, expenses, announcements, livestream links, and a private wedding website.

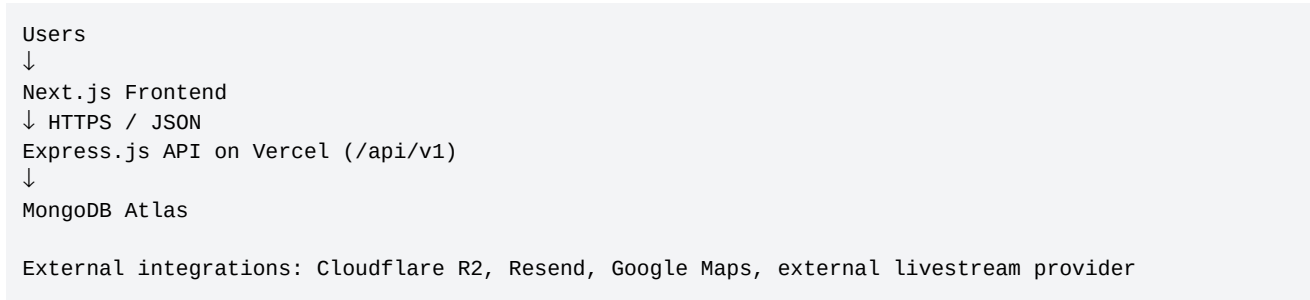
The product is private and invitation-only. Family users have Admin or Manager accounts. Guests do not create accounts or passwords; they access permitted information using unique invitation links or QR codes.

3. System Actors

Actor	Responsibilities
Admin	Critical wedding controls, users, invitations, photo deletion, publishing, and protected settings.
Manager	Daily wedding operations, events, tasks, vendors, expenses, and other permitted management activities.
Guest	View invited event information, submit event-level RSVP, view the wedding gallery, and access permitted livestreams.
External services	MongoDB Atlas, Cloudflare R2, Resend, Google Maps links, and an external livestream provider.

4. High-Level Architecture

The frontend and backend are deployed as separate Vercel projects. The public application uses a shared domain with controlled routing so API requests use the /api/v1 path.



The frontend must not directly access MongoDB, Resend credentials, or private Cloudflare R2 credentials.

5. Deployment Architecture

Component	Deployment / Responsibility
Frontend project	Next.js application deployed on Vercel.
Backend project	Single Express.js application entry point deployed through Vercel serverless execution.
Routing	Controlled rewrite/routing from the shared public domain to /api/v1 endpoints.
Database	MongoDB Atlas, with separate development, test/preview, and production databases or environments.
Object storage	Private Cloudflare R2 bucket for original photos.
Email	Resend API called directly by the Express backend in small batches.

6. Backend Structure

The selected structure is layer-based with clear module boundaries. The application remains a modular monolith, not a collection of microservices.

```
src/  
  config/  
  routes/  
  controllers/  
  services/  
  models/  
  validations/  
  middleware/  
  integrations/  
    resend/  
    cloudflare-r2/  
  app.ts  
  server.ts
```

7. Backend Modules

Module	Main responsibilities
Authentication & Authorization	Registration, login, JWT cookies, password reset, roles, and authentication invalidation.
Wedding	Wedding details, location, coordinates, dates, membership, status, and archiving.
Event	Event dates, time, venue, map link, descriptions, access scope, and archiving.
Guest & Invitation	Invitation tokens, QR codes, expiry, revocation, guest assignments, and guest access.
RSVP	Event-level Yes/No responses, duplicate prevention, and reminder eligibility.
Photo & Gallery	R2 upload URLs, metadata validation, private delivery URLs, and limited downloads.
Task	Task creation, assignees, due dates, status, and event association.
Vendor	Vendor records, categories, status, and task linking.
Expense	Planned/actual amounts, categories, payment status, notes, and summaries.
Wedding Website	Templates, draft/preview/publish workflow, sections, language, and public presentation.
Livestream & Announcements	Event-specific livestream links and wedding announcements.
Notifications	Invitation emails, RSVP reminders, event reminders, and critical change emails.

8. Request Processing Flow

Standard family-user request flow:

- The user opens the Next.js application.
- The frontend sends a request to a versioned REST endpoint.
- Authentication middleware verifies the access JWT from the HttpOnly cookie.
- Authorization middleware performs broad role checks.
- Validation middleware validates request input with Zod.
- The controller delegates business logic to a service.
- The service applies wedding ownership, membership, invitation, and business rules.
- The Mongoose model reads or writes MongoDB Atlas.
- The API returns the standard response envelope.

Guest invitation flow:

- The guest opens a unique invitation link or scans a QR code.
- The backend hashes the invitation token and compares it with the stored hash.
- The backend checks expiry, revocation, and event permissions.
- The backend issues a short-lived HttpOnly guest cookie.
- Protected requests revalidate invitation status so revocation becomes effective on the next validation.

9. API Conventions

All API endpoints are versioned under /api/v1 and return JSON.

Success response:

```
{
  "success": true,
  "data": {}
}
```

Error response:

```
{
  "success": false,
  "error": {
    "code": "EVENT_NOT_FOUND",
    "message": "Event not found"
  }
}
```

- Centralized error-handling middleware.
- Zod validation at the API boundary.
- Mongoose validation as an additional safeguard.
- Page-number pagination with stable sorting where collections are listed.
- Indexes based on actual query patterns.
- Idempotency keys for selected critical operations.

10. Authentication and Security

Area	Decision
Tokens	Short-lived access JWT plus refresh JWT.
Storage	Both tokens are stored in secure HttpOnly cookies.
Access token	15-minute lifetime; cookie path /.
Refresh token	7-day lifetime; cookie path /api/v1/auth.
Cookie settings	SameSite Lax initially, Secure in production, host-only cookies, Domain omitted.
Password hashing	Argon2id with deployment-appropriate cost configuration.
Refresh token storage	Stateless; no refresh-token records in MongoDB.
CSRF approach	Origin validation for state-changing requests plus SameSite controls.
Rate limiting	MongoDB-backed atomic counters with expiry for sensitive endpoints.
Authorization	Role middleware plus service-level wedding and invitation checks.

Stateless refresh JWTs cannot be individually revoked immediately. The design should include secure secret management, short token lifetimes, and user-level invalidation mechanisms such as a token version or authentication invalidation field when required.

11. Guest Invitation Security

- Invitation tokens are random opaque values and only their hashes are stored.
- Guest links can technically be forwarded; V1 does not provide identity verification or OTP.
- Invitation expiration is required and controlled by the Admin.
- A short-lived guest cookie is used after successful invitation validation.
- Guest requests are checked against invitation status and event scope.
- Guests see only the events they are invited to.
- All valid invited guests can view the wedding-level gallery.

12. Photo and Cloudflare R2 Architecture

V1 intentionally avoids image resizing, thumbnails, compression, and background image-processing pipelines. Original photos are stored in private Cloudflare R2.

- Frontend requests a presigned upload URL from Express.
- Backend validates user or guest permissions and file details.
- Backend generates a temporary R2 presigned URL.
- Frontend uploads directly to R2.
- Frontend notifies the backend after upload.
- Backend performs basic file validation and server-side R2 object metadata checks.
- Photo metadata is stored in MongoDB.
- Photo delivery uses short-lived authorized signed URLs or equivalent protected CDN delivery.
- Maximum 10 photos per upload and maximum 10 MB per photo.

- Accepted formats: JPG, PNG, and WEBP. Video uploads are not supported.
- Only Admin can delete photos; deletion follows the approved soft-delete/archive policy where applicable.
- V1 supports limited synchronous ZIP downloads with a reasonable bulk-download limit.

13. Email and Notifications

- Resend API is used for email delivery.
- Emails are sent directly from the Express backend in small batches.
- No Redis, BullMQ, or separate worker is introduced in V1.
- Basic email status may be stored in MongoDB if tracking is needed.
- SENT means Resend accepted the email request.
- Bounded retries and failure status are used for appropriate operations.
- One RSVP reminder is sent to non-responders.
- Event reminders are sent 24 hours before the event where scheduling support is available.
- Critical event changes can trigger notification emails.

This lightweight approach reduces complexity but provides weaker retry and scheduling guarantees than a dedicated queue. A queue can be added later if reliability or volume requires it.

14. Data and Lifecycle Rules

- Events are archived or marked inactive rather than permanently deleted.
- Archived data is preserved and may be viewable through Admin-controlled access.
- Wedding data is retained indefinitely in V1 unless a later policy is introduced.
- The system should enforce reasonable wedding, invitation, and photo/storage limits.
- All critical operations should check wedding membership, role, and relevant scope.

15. Observability and Reliability

- Use simple structured application logs through console/runtime logging.
- Never log passwords, JWTs, invitation tokens, presigned URLs, or other secrets.
- Use request IDs and safe error responses where practical.
- Apply layered timeouts: platform/runtime, Express processing, MongoDB operations, and external APIs.
- Avoid blind retries, especially for non-idempotent writes.
- Return controlled 503 responses for temporary dependency failures.
- Use unit tests and API integration tests for critical flows.
- Use React Testing Library and selected end-to-end tests for important frontend flows.

16. Backup and Disaster Recovery

V1 uses a practical recovery approach rather than multi-region infrastructure.

- Document MongoDB Atlas backup configuration and retention.
- Document Cloudflare R2 backup and object-recovery considerations.
- Define restoration procedures and responsible team members.
- Perform periodic restore testing before production launch.
- Document recovery steps and review them after major architecture changes.

17. Development and Deployment Workflow

- Use separate repositories: make-my-wedding-frontend and make-my-wedding-backend.
- Maintain development, test/preview, and production environments.
- Use Git-based deployment with lint, type-check, and test gates.
- Manage production secrets through Vercel environment variables/secrets.
- Use separate databases or isolated environments before onboarding real production users.
- Verify Vercel routing, cookie behavior, preview environments, and error handling during implementation.

18. Design Principles and Non-Goals

Core principles:

- Backend is the security boundary.
- Controllers remain thin; services own business logic.
- Keep integrations isolated.
- Use modular boundaries without microservices.
- Prefer simple V1 implementation over premature infrastructure.

Explicit V1 non-goals:

- No microservices.
- No Redis cache or queue initially.
- No separate background worker initially.
- No custom livestream infrastructure.
- No image-processing pipeline.
- No public wedding search.
- No guest accounts with passwords.
- No accommodation or transportation management in V1.

19. Next Architecture Phase

The next phase is Database Design. It will define MongoDB collections, document relationships, embedded versus referenced data, indexes, soft-archive fields, and consistency rules.